

Online Retail II — Business Findings & Recommendations

A business-intelligence & predictive-analytics review of two years of transactional sales data, prepared by Berfin · Data Analyst Portfolio Project · September 2026

£20.96M	1,061,163	5,942	£19.90	0.816
Net revenue (2 yrs, 43 countries)	Invoice lines analysed	Customers profiled	Average order value	Churn model ROC-AUC

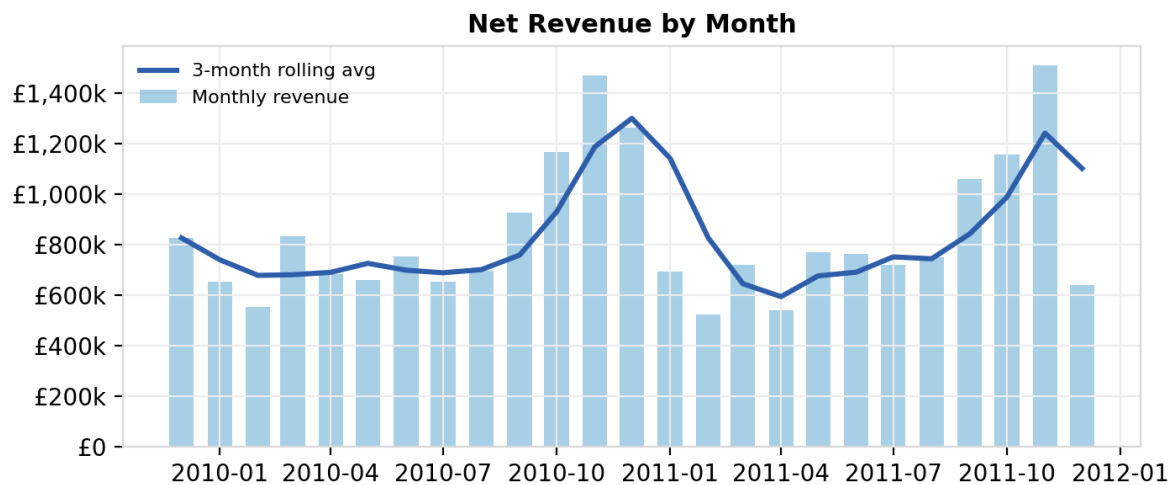
Why I built this

I wanted a portfolio project that goes beyond a single descriptive notebook and instead mirrors how a real data/analytics team works end to end: a proper SQL data warehouse, a live external data feed, exploratory and business-intelligence analysis in Python, a supervised machine-learning model tied to a concrete business decision, and — the part I'm writing right now — a short findings report that translates all of it into plain business language. The dataset is the UCI "Online Retail II" set: ~1.07 million invoice lines from a UK-based online retailer of gift and homeware items, December 2009 to December 2011.

What I did

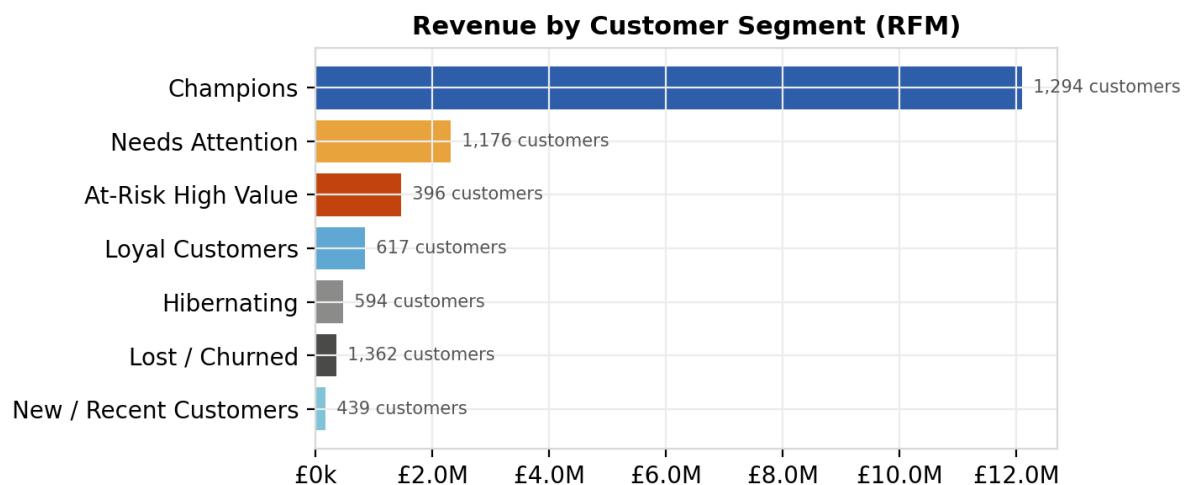
- **Built a PostgreSQL data warehouse** — a cleaned star schema (fact_sales + dim_customer/product/date/country) from the raw source file, with explicit rules for cancellations, guest checkouts, and bad-debt adjustment postings (sql/00–01).
- **Wrote 15+ SQL analyses** directly against that warehouse — KPI summaries, monthly/quarterly trends, country concentration, RFM customer segmentation, cohort retention, customer lifetime value, and market-basket (cross-sell) analysis (sql/02–03).
- **Integrated a live external API** — pulling historical and current GBP→USD/EUR/TRY exchange rates to convert revenue into the currencies our non-UK stakeholders actually think in (scripts/live_currency_api.py).
- **Ran a Python EDA/BI notebook** with 8 charts, each followed by an explicit business takeaway rather than a bare description of the data (notebooks/01).
- **Trained and evaluated a churn-prediction model** (Logistic Regression + Random Forest) on a leakage-safe 90-day holdout split, plus a secondary customer-value regression, and turned the output into a ranked, actionable retention target list (notebooks/02).

What I found



Net revenue by month, with a 3-month rolling average. December 2011 is a partial month — the extract stops on Dec 9 — not a real demand collapse.

- 1. Growth is real, but seasonal and increasingly retention-driven.** Revenue builds steadily into a September–November peak each year and troughs in Jan/Feb. The share of monthly revenue coming from returning (not new) customers climbs from roughly two-thirds early in the dataset to 85–95% later on — the business is leaning more and more on the customers it already has.
- 2. The UK dominates, but the real opportunity is the next tier down.** A large majority of revenue comes from the UK alone; EIRE, Germany, France and the Netherlands are the markets with proven repeat demand, and are a better next-expansion bet than a blank-slate new country.
- 3. Retention has a sharp month-1 cliff.** Cohort analysis shows most of the drop-off in repeat purchasing happens in the first 30 days after a customer's first order, in every monthly cohort I looked at — this is the single highest-leverage point for a lifecycle/onboarding campaign to target.



RFM (Recency/Frequency/Monetary) segmentation, computed in SQL and validated against the churn model in the notebook.

- 4. Revenue is heavily concentrated in a small "Champions" segment** (1,294 of 5,942 customers, ~69% of total revenue). The **396-customer "At-Risk High Value" segment** — proven high spenders who have gone quiet — is the clearest, highest-ROI retention target.
- 5. The churn model works, and it agrees with the segmentation.** A Random Forest classifier trained on a leakage-safe 90-day holdout (features from before the cutoff, label from after it) reaches ROC-AUC = 0.816 — recency and order frequency are the strongest signals. Predicted risk rises monotonically from Champions to Lost/Churned, cross-validating the SQL-based segmentation and the ML model against each other. Ranking customers by *probability* × *historical value* (not probability alone) produced a 25-name target list exported for a retention campaign (exports/churn_risk_list.csv).

6. Clear, low-risk cross-sell pairs exist and are not currently merchandised together — mostly colour/pattern variants of the same product (matching cutlery sets, egg cups, cake stands) with purchase-lift scores of 200–400×, i.e. customers who buy one are 200–400 times more likely than chance to also buy its pair in the same order.

What I recommend

- **Launch a first-30-days lifecycle campaign** (welcome flow + a modest second-order incentive) — this is where the retention curve is steepest, so it's the highest-leverage fix available.
- **Run a proactive win-back campaign on the 25-name churn-risk list** before those customers fully lapse, prioritised by expected revenue at risk rather than by churn probability alone.
- **Feature the identified product pairs together** on product pages and at cart/checkout — a low-effort, high-confidence conversion lever versus speculative new bundling.
- **Prioritise EIRE, Germany, France and the Netherlands** for international investment over new-market entry, since they already show repeat (not one-off) demand.
- **Report exec revenue in period-correct multi-currency terms** (each month converted at that month's own rate, not one static rate) — the live-API pipeline built for this project already produces that table.

Limitations & next steps

This is historical data (through Dec 2011) from a single UK retailer, so absolute figures are illustrative rather than current — the methodology, not the specific numbers, is the deliverable. The churn model's 90-day definition is a reasonable default but should be tuned against the business's actual reorder cycle; the secondary revenue-regression model is noisy enough that I'd trust it only for relative ranking, not as a financial forecast. A natural next step is a live Power BI dashboard on top of the star schema built here, which I built together with a Power BI Desktop session — see the README for that walkthrough and screenshot.

Full SQL, Python notebooks, the live-API script and the data warehouse build scripts are in the accompanying GitHub repository. — Berfin